Solve the Poisson Equation with PINN

Qimu Luo

June 19, 2024

Contents

1 Introduction **2**

2 Poisson Equation **2**
2.1 Three Types of Boundary Conditions . . . 3
2.1.1 Dirichlet boundary conditions . . . 3
2.1.2 Neumann boundary conditions . . . 3
2.1.3 Robin boundary conditions . . . 4

3 PINN **4**
3.1 Interpretation . . . 4
3.2 Structure . . . 5

4 The Weights of the Boundary Loss **5**

5 Comparison Under Different Circumstances **7**
5.1 Dirichlet . . . 7
5.1.1 Iterations (Red for real solution; Blue for prediction) . . . 7
5.1.2 Training data sets . . . 8
5.1.3 Architecture . . . 9
5.1.4 Weights . . . 9
5.2 Neumann . . . 10
5.2.1 Iterations (Red for real solution; Blue for prediction) . . . 10
5.2.2 Training data sets . . . 11
5.2.3 Architecture . . . 12
5.2.4 Weights . . . 13
5.3 Robin . . . 14
5.3.1 Iterations (Red for real solution; Blue for predicion) . . . 14
5.3.2 Training data sets . . . 14
5.3.3 Architecture . . . 16
5.3.4 Weights . . . 17

6 Confusions of Details **17**

7 Limitations **18**

1 Introduction

This report describes a method for solving the Poisson equation with three boundary conditions using physical-informed neural networks. It is divided into three main sections. The first section briefly introduces the theory. The second part compares the experimental results obtained under different circumstances. Finally, the third section summarizes the confusion that was solved over the past two months.

2 Poisson Equation

According to Raissi et al. [2], the nonlinear PDEs take the general form:

$$u_t + N[u; \lambda] = 0, x \in \Omega, t \in [0, T] \tag{1}$$

- $u(t, x)$ denotes the hidden solution
- $N[\cdot; \lambda]$ is a nonlinear operator parametrized by λ, which describes noisy measurements of a generic dynamic system.
- Ω is a subset of $\mathbb{R}^D$

As a type of PDE, the Poisson equation only considers $\mathbf{x}$, which refers to the space variable and ignores the time variable t. Therefore, it only considers the boundary conditions without the initial condition defined by the time variable t. It is also a special case of elliptic partial differential equations as a generalization of the Laplace equation. The basic form is:

$$\Delta u = f, \quad x \in \Omega \tag{2}$$

- Δ: a Laplace operator, serving as the role of N in equation 1
- f: given as the right-hand side function
- u: terms we need to solve

Here only a two-dimensional Poisson equation considering three types of boundary conditions will be covered.

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \tag{3}$$

2.1 Three Types of Boundary Conditions

To define the equation comprehensively, we need to equip it with boundary conditions consisting of three types: Dirichlet, Neumann, and Robin, which can be demonstrated in a general form as follows. If we define the boundary condition as $Ay + By' = C$, then:

$$\begin{cases} A \neq 0, B = 0 \Rightarrow \text{Dirichlet} \\ A = 0, B \neq 0 \Rightarrow \text{Neumann} \\ A \neq 0, B \neq 0 \Rightarrow \text{Robin} \end{cases} \tag{4}$$

2.1.1 Dirichlet boundary conditions

1. Interpretation: this boundary condition (first-type) specifies the value of the solution itself on the boundary directly.

$$u(\mathbf{x}) = u_{\partial}(\mathbf{x}), \quad \forall \mathbf{x} \in \partial\Omega \tag{5}$$

2. Example: the example we used here is the analytical solution $u(x, y) = \sin \pi x \sin \pi y$ with the zero-boundary condition on a unit square $\Omega = [0, 1]^2$

$$\begin{cases} \Delta u(x, y) = -2\pi^2 \sin \pi x \sin \pi y \quad (x, y) \in (0, 1)^2 \\ u(1, y) = 0 \quad y \in (0, 1) \\ u(0, y) = 0 \quad y \in (0, 1) \\ u(x, 1) = 0 \quad x \in (0, 1) \\ u(x, 0) = 0 \quad x \in (0, 1) \end{cases} \tag{6}$$

2.1.2 Neumann boundary conditions

1. Interpretation: the condition (second-type) specifies the values of the derivative applied at the boundary of the domain, taking the form:

$$\frac{\partial u}{\partial \mathbf{n}}(\mathbf{x}) = f(\mathbf{x}) \quad \forall \mathbf{x} \in \partial\Omega \tag{7}$$

   - $\mathbf{n}$ denotes the exterior normal to the boundary $\partial\Omega$
   - f is a given function

   The left-hand side normal directive is defined as:

$$\frac{\partial u}{\partial \mathbf{n}}(\mathbf{x}) = \nabla u(\mathbf{x}) \cdot \hat{\mathbf{n}}(\mathbf{x}) \tag{8}$$

   - $\nabla u(\mathbf{x})$ denotes the gradient vector of $u(\mathbf{x})$
   - $\hat{\mathbf{n}}$ means the unit normal vector of $\mathbf{n}$
   - $\cdot$ represents the inner product operator

   Note: since this boundary condition requires the derivative on the boundaries, it must be sufficiently smooth such that the normal derivative can exist.

2. Example: the two-dimensional Poisson equation is defined as:

$$\begin{cases} \Delta u(x,y) = 2(x^4(3y-2) + x^3(4-6y) + x^2(6y^3 - 12y^2 + 9y - 2) \\ -6x(y-1)^2 y + (y-1)^2 y) \quad (x,y) \in (0,1)^2 \\ \partial_{\vec{n}} u(0,y) = 0 \quad y \in [0,1] \\ \partial_{\vec{n}} u(1,y) = 0 \quad y \in [0,1] \\ \partial_{\vec{n}} u(x,1) = 0 \quad x \in [0,1] \\ u(x,0) = 0 \quad x \in [0,1] \end{cases} \tag{9}$$

The solution can be solved analytically:

$$u_{\text{true}}(x,y) = x^2(x-1)^2 y(y-1)^2 \tag{10}$$

2.1.3 Robin boundary conditions

1. Interpretation: the condition (third-type) is a linear combination of Dirichlet and Neumann's conditions shown in the boundary mixture 4. Therefore, we can combine 5 and 7 to form the condition:

$$au(\mathbf{x}) + b\frac{\partial u}{\partial \mathbf{n}}(\mathbf{x}) = g, \quad \forall \mathbf{x} \in \partial\Omega \tag{11}$$

   - a and b are given as constants or functions more generally
   - g denotes a given function

2. Example: we replace the boundary condition of the upper side $(x, 1)$ in the Neumann example 9 with the robin boundary condition then the example contains three types of boundary conditions.

$$\begin{cases} \Delta u(x,y) = 2(x^4(3y-2) + x^3(4-6y) + x^2(6y^3 - 12y^2 + 9y - 2) \\ -6x(y-1)^2 y + (y-1)^2 y) \quad (x,y) \in (0,1)^2 \\ \partial_{\vec{n}} u(0,y) = 0 \quad y \in [0,1] \\ \partial_{\vec{n}} u(1,y) = 0 \quad y \in [0,1] \\ \partial_{\vec{n}} u(x,1) + u(x,1) = 0 \quad x \in [0,1] \\ u(x,0) = 0 \quad x \in [0,1] \end{cases} \tag{12}$$

3 PINN

3.1 Interpretation

PINN refers to the physical-informed neural networks, which are employed to approximate the PDE solutions utilizing the Universal Approximation Theorem of neural networks. Specifically, the automatic differentiation technology is used to compute the approximation of PDE. Depending on the presence or absence of λ in equation 1, the problems solved by PINNs are divided into two categories:

- Data-driven solution: fixed model parameters λ

- Data-driven discovery: learning model parameters λ that best describe the observed data

Here we ignore the noise factor and only consider the data-driven solution problems without λ. To approximate the solution, we define the residual $f(\mathbf{x})$ as:

$$f := u + N[u] = 0 \tag{13}$$

$u(\mathbf{x})$ is approximated by a neural network, which can be differentiated using automatic differentiation provided by PyTorch. Based on Rassi et al [2]'s method, the shared parameters between the neural networks $u(x, y)$ and $f(x, y)$ can be learned by minimizing the mean squared error:

$$\text{MSE} = MSE_u + MSE_f \tag{14}$$

where

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} \left| u\left(x_u^i, y_u^i\right) - u^i \right|^2, \tag{15}$$

and

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| f\left(x_f^i, y_f^i\right) \right|^2. \tag{16}$$

- $\{x_u^i, y_u^i\}_{i=1}^{N_u}$ denote the boundary training data on $u(x, y)$
- $\left\{x_f^i, y_f^i\right\}_{i=1}^{N_f}$ specify the collocations points for $f(x, y)$

Additionally, the L_2 error is employed to evaluate the performance of the model created by the neural network. The whole training process of PINNs is summarized in figure 1.

3.2 Structure

We use the feedforward neural network with 3 hidden layers and 64 neurons to solve the problem. The activation function is Tanh, the optimizer is Adam and the learning rate is $1e-3$.

4 The Weights of the Boundary Loss

PINN-related information on Wikipedia [4] mentions one of the PINNs limitations is that PINNs easily fail to train. One of the reasons behind the failure of the regular PINNs is the soft-constraining of boundary conditions which pose multi-objective optimization problems. Therefore, manually weighing the loss terms is needed for the neural network to optimize, which will also be demonstrated in 5 section. Moreover, in our experiments, we find that the appropriate weights for the loss function seem to vary for each equation and even under different circumstances for the same equation. Wang, Yu, and Perdikaris[3] provide a possible explanation from the perspective of the eigenvalue of the NTK (Neural Tangent Kernel), which describes the rate of convergence of the loss. They've obtained several conclusions from their experiments:

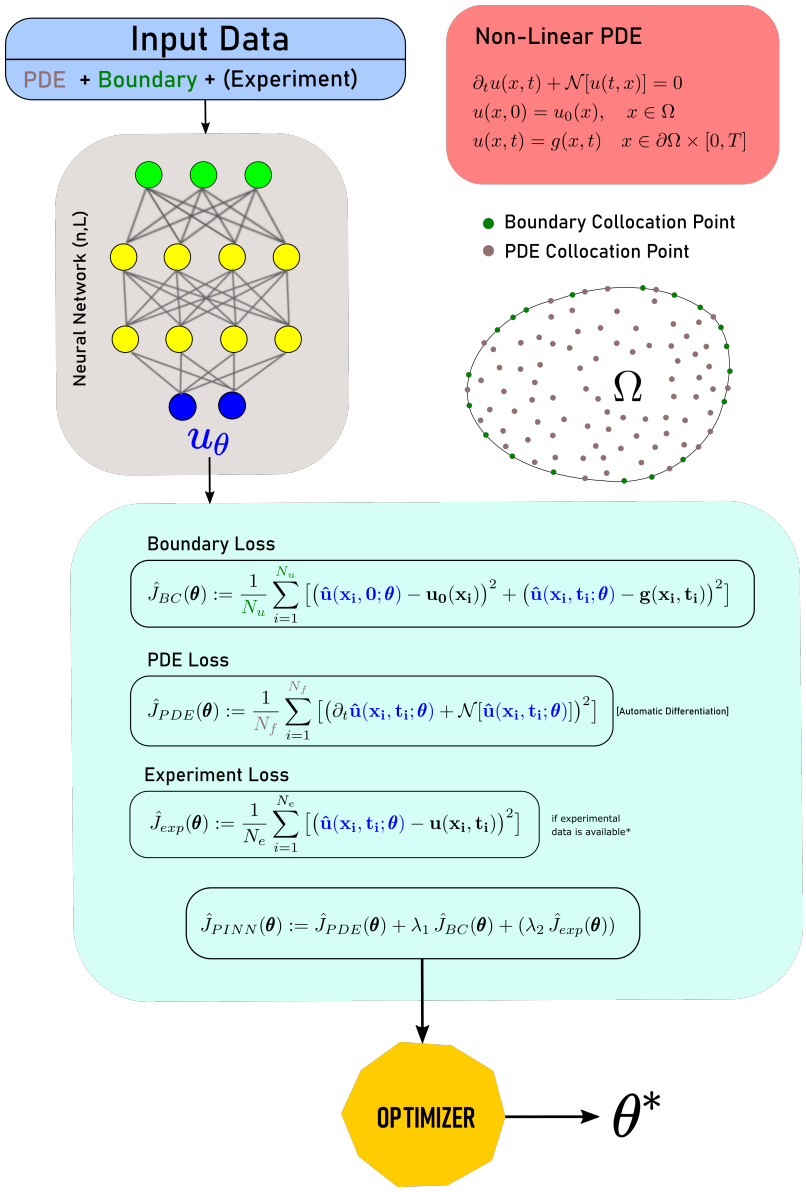


Figure 1: PINN schematic

1. Fully connected PINNs not only suffer from spectral bias but also from a remarkable discrepancy of convergence rate in the different components of the loss function.

2. The eigenvalues of the inner loss are much greater than the boundary loss. As a consequence, the PDE residual in equation 16 converges much faster than fitting the boundary conditions in equation 15, which may prevent the network from approximating the correct solution.

3. The discrepancy between different loss terms is one of the key fundamental reasons why PINNs can often fail to train and reach accurate predictions.

4. The novel adaptive training strategy proposed by them can enhance the trainability and predictive accuracy of PINNs.

Although the method is only proved in the infinite-width neural network, it indeed provides theoretical support to our adjustment by increasing the weight of the poor training condition which also serves as a hard constraint of the boundary loss function.

5 Comparison Under Different Circumstances

We've compared the training outcomes under different circumstances to figure out which factor may influence the result greatly and find out the optimal combination for the model. To compare different situations, one of the conditions below is changed and fixed by the others.

- Architecture of neural network: the same as section 3.2
- The number of the training data set: $N_b = 100$, $N_f = 2500$
- Iterations: 10000
- Weights:
  - Dirichlet example: $D_{\text{weight}} = 50$
  - Neumann example: $D_{\text{weight}} = 250$, $N_{\text{weight}} = 10$
  - Robin example: $D_{\text{weight}} = 50$, $N_{\text{weight}} = 15$, $R_{\text{weight}} = 10$

Note: For each iteration, we consider the least L_2 error as representative of the error at that iteration.

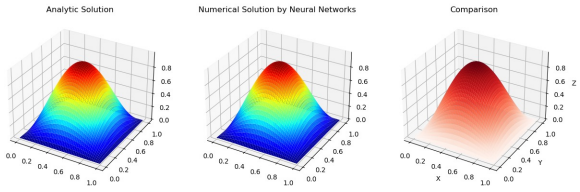
5.1 Dirichlet

5.1.1 Iterations (Red for real solution; Blue for prediction)

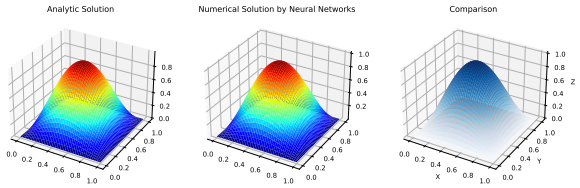
The example is the most well-behaved. Results of iterations at 5000, 10000, and 50000 are compared.

1. Compare numerical solution and analytical solution: shown in figure 2. After only 5000 iterations, it is difficult to distinguish the difference between the numerical and analytical solutions with the naked eye, similar to the results after 10000 as well as 50000 iterations.
2. Compare PDE loss: it is obvious in figure 3 that the PDE loss is decreasing from less than $1e-2$ to less than $1e-4$ as the iteration increases from 5000 to 50000, suggesting that the iterations assist in improving the training results.
3. Compare L_2 error: shown in figure 4, the L_2 error decreases as the number of iterations increases. However, the magnitude of the L_2 error oscillations increases at the same time and it is very difficult to reach the $1e-3$ finally.

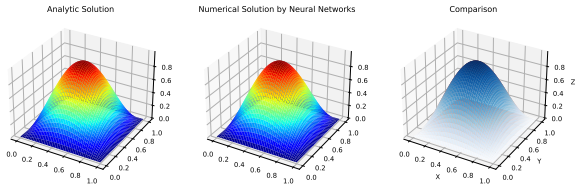
The Table 1 records the results of different iterations.



(a) iteration = 5, 000

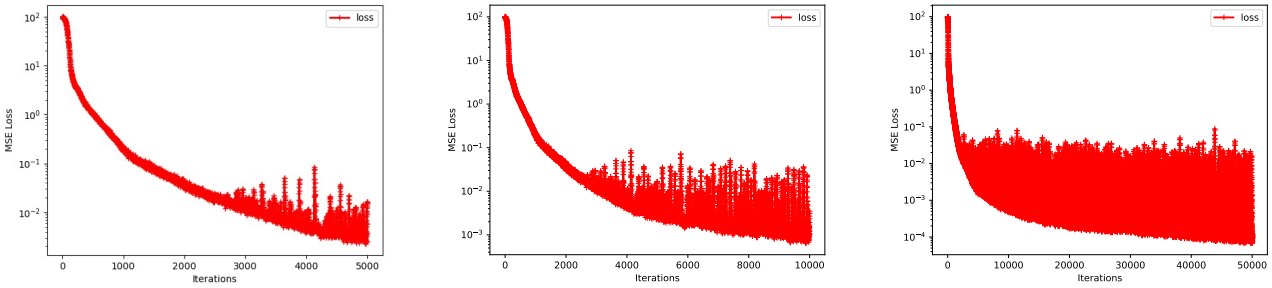


(b) iteration = 10, 000



(c) iteration = 50, 000

Figure 2: Comparison of Numerical and Analytical Solution

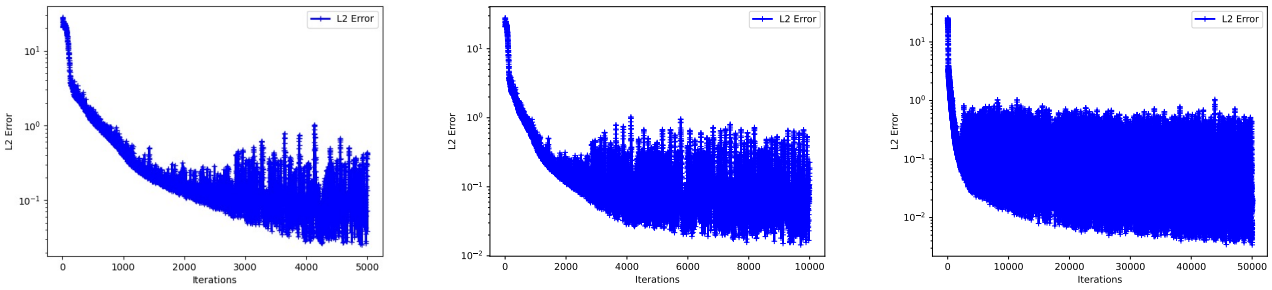


(a) iteration = 5, 000 (b) iteration = 10, 000 (c) iteration = 50, 000

Figure 3: Comparison of PDE Loss

5.1.2 Training data sets

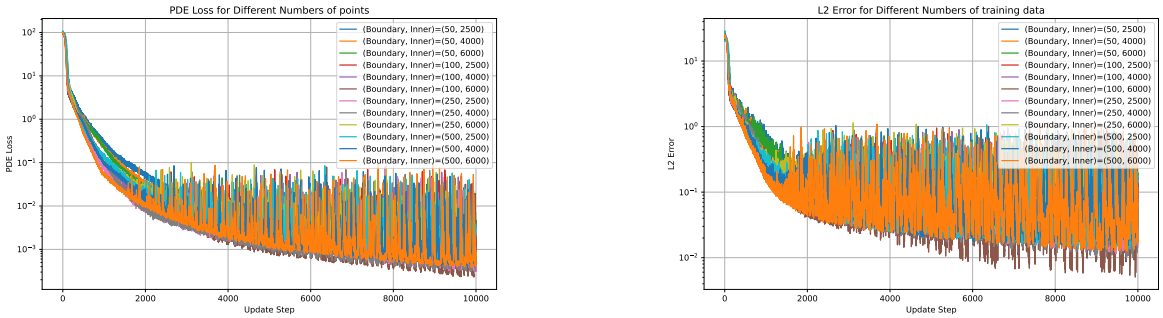
We then explored whether the size of the training set would have an impact on the results, and recorded the PDE losses in figure 5a and L_2 errors in figure 5b for different combinations of boundary and interior points. As obtained from the figure 5, the oscillation of the losses for



(a) iteration = 5, 000 (b) iteration = 10, 000 (c) iteration = 50, 000

Figure 4: Comparison of L_2 Error

different sizes of training data tends to be the same. Therefore, the size of the training set does not affect the results significantly for this example.



(a) PDE loss of different numbers of points (b) L_2 error of different numbers of points

Figure 5: Compare Losses of Different Sizes of Data-Sets

5.1.3 Architecture

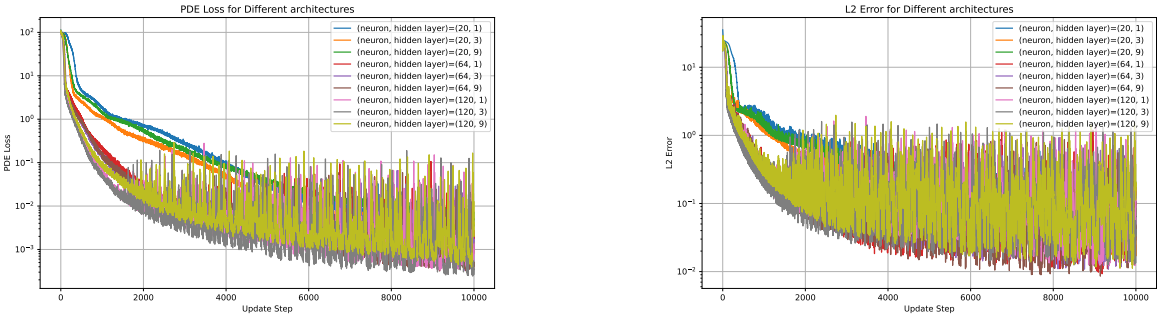
Finally, we vary the number of neurons as well as the number of hidden layers to compare their results. It can be seen from the figure 6 that when the number of neurons is smaller at 20, the loss is harder to decrease at the beginning of the iteration. However, as the number of iterations increases, the results of the training tend to be the same. Therefore, the number of neurons should not be too small if a smaller number of iterations is desired.

5.1.4 Weights

Due to the better property of this example 6, which already demonstrates good fitting ability with $D_{\text{weight}} = 50$ on the boundary, no other comparisons will be made.

Iterations	Error	Runtime
10,000	2.26E-01	88.113
20,000	9.79E-02	172.963
30,000	2.68E-02	235.430
40,000	6.69E-03	325.959
50,000	1.73E-02	374.328

Table 1: Comparison of Iterations



(a) PDE loss of different architectures

(b) L_2 error of different architectures

Figure 6: Compare Losses of Different Architectures

5.2 Neumann

5.2.1 Iterations (Red for real solution; Blue for prediction)

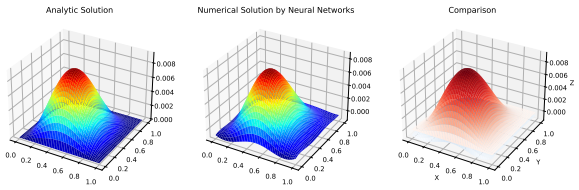
Following the same procedures as Dirichclet, the results of iterations at 10000, 30000, and 50000 are compared.

1. Compare numerical solution and analytical solution: shown in figure 7. It can be seen that the effect of the number of iterations has a large impact on this example, the boundaries especially the Dirichlet condition on the downside $(x, 0)$ are difficult to train and the model does not show fit well until 50000 iterations.

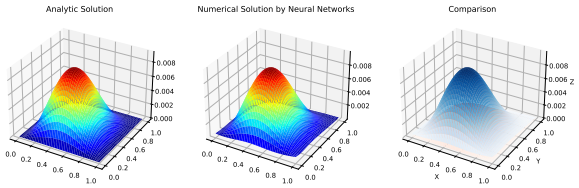
2. Compare PDE loss: it is obvious in figure 8 that the PDE loss is decreasing form less than $1e-3$ to less than $1e-4$ as the iteration increases from 10000 to 50000. However, it is harder to drop below $1e-4$.

3. Compare L_2 error: shown in figure 9, the L_2 error decreases as the number of iterations increases, however, the magnitude of L_2 error oscillation increases at the same time and it is very difficult to reach the $1e-3$ until the iteration $= 50000$. This shows that the nature of the model is not very stable.

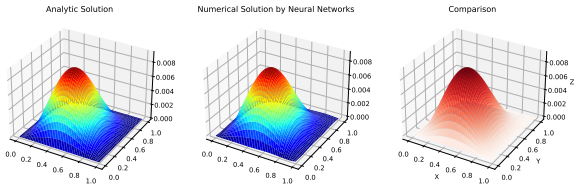
The Table 2 records the results of different iterations.



(a) iteration = 10, 000

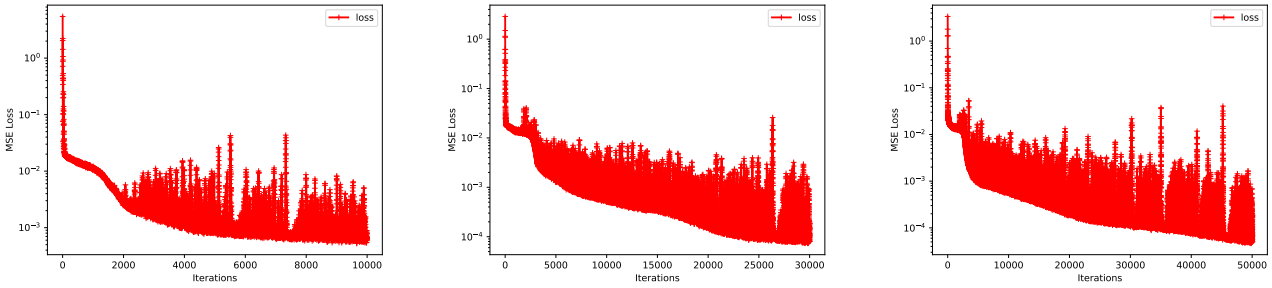


(b) iteration = 30, 000



(c) iteration = 50, 000

Figure 7: Comparison of Numerical and Analytical Solution



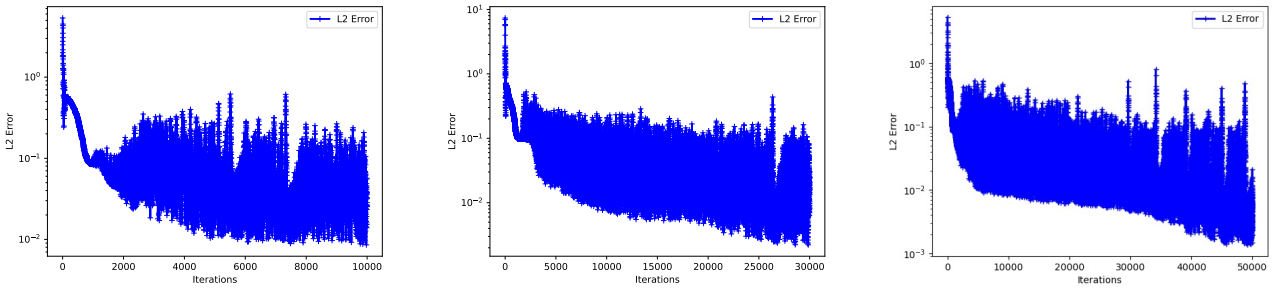
(a) iteration = 10, 000 (b) iteration = 30, 000 (c) iteration = 50, 000

Figure 8: Comparison of PDE Loss

5.2.2 Training data sets

We then explore whether the size of the training set will have an impact on the results, and record the losses in figure 10 for different combinations of boundary and interior points.

1. PDE loss in figure 10a: the combination ($N_b = 100, N_f = 4000$) demonstrates the best



(a) iteration = 10, 000 (b) iteration = 30, 000 (c) iteration = 50, 000

Figure 9: Comparison of L_2 Error

Iterations	Error	Runtime
10,000	3.18E-02	83.995
20,000	2.39E-02	163.783
30,000	2.09E-02	252.042
40,000	8.80E-03	323.1
50,000	6.05E-03	402.2

Table 2: Comparison of Iterations

outcome to minimize the PDE loss. Meanwhile, comparing the PDE loss of ($N_b = 500, N_f = 6000$), we can obtain the conclusion that expanding the training set does not necessarily lead to the greatest decrease in loss, and there is also the possibility of falling into a local optimal point.

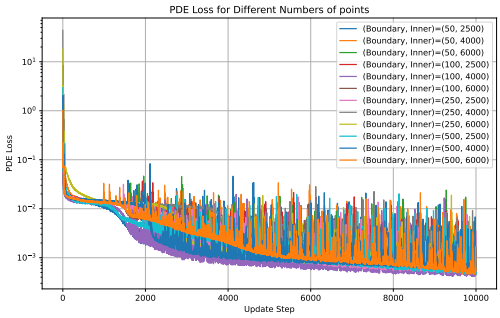
2. L_2 error in figure 10b: the oscillation of the losses for different sizes of training data-set tend to be the same. Therefore, the size of the training set does not affect the performance of the model significantly for this example.

5.2.3 Architecture

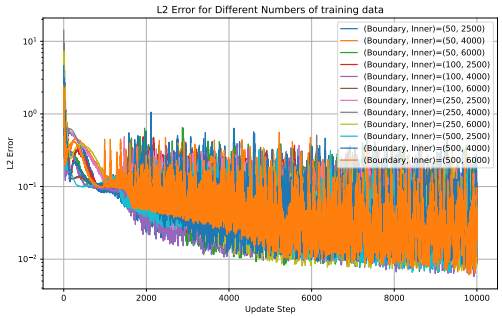
Finally, we vary the number of neurons as well as the number of hidden layers to compare their results.

1. The PDE losses in figure 11a: when the number of neurons is smaller at 20, the loss is harder to decrease until iteration at 6000.

2. The L_2 error in figure 11b: the oscillatory trends of the L_2 errors tend to be similar, suggesting that the architecture does not affect the performance of the model.

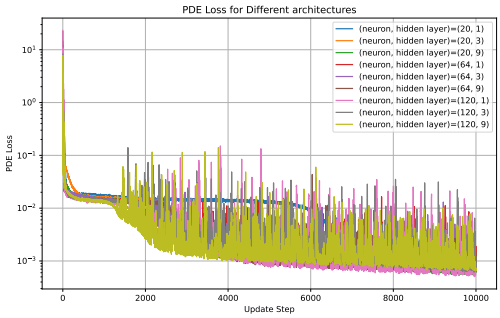


(a) PDE loss of different numbers of points

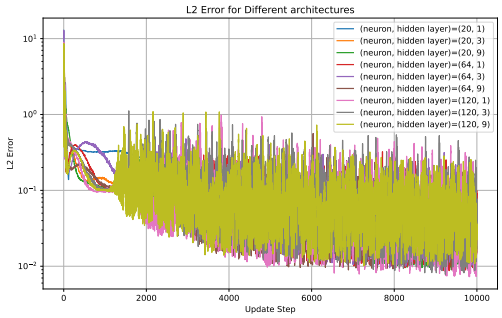


(b) L_2 error of different numbers of points

Figure 10: Compare Losses of Different Sizes of Data-Set



(a) PDE loss of different architectures

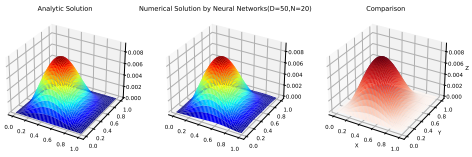


(b) L_2 error of different architectures

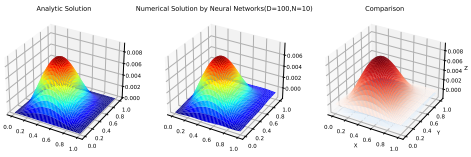
Figure 11: Compare Losses of Different Architectures

5.2.4 Weights

Different combinations of weights are tested to evaluate the performance at the iteration = 40000, and Table 3 records the results, which are sorted in order of error from smallest to largest. It is more intuitive to compare the numerical and analytical solution of ($D_{\text{weight}} = 50, N_{\text{weight}} = 20$) and ($D_{\text{weight}} = 100, N_{\text{weight}} = 10$). The former figure 12a almost fits all boundaries at 40000 iteration but the latter figure 12b almost fails to fit the boundaries.



(a) ($D_{\text{weight}} = 50, N_{\text{weight}} = 20$)



(b) ($D_{\text{weight}} = 100, N_{\text{weight}} = 10$)

Figure 12: Comparison of Numerical and Analytical Solution

Weights(Dirichlet, Neumann)	Error	Runtime
(50, 20)	4.32E-03	322.185
(300, 10)	5.65E-03	328.274
(50, 10)	8.82E-03	324.827
(100, 20)	9.12E-03	313.862
(250, 20)	1.55E-02	330.592
(300, 20)	1.90E-02	310.191
(250, 10)	2.97E-02	317.182
(100, 10)	5.82E-02	322.581

Table 3: Comparison of Weights of Boundaries

Iterations	Error	Runtime
10,000	8.62E-02	83.249
20,000	8.24E-02	160.927
30,000	6.41E-03	242.205
40,000	1.57E-02	323.179
50,000	1.00E-02	390.774

Table 4: Comparison of Iterations

5.3 Robin

5.3.1 Iterations (Red for real solution; Blue for predicion)

Following the same procedures as above, the results of iterations at 10000, 30000, and 50000 are compared.

1. Compare numerical solution and analytical solution: shown in figure 13. It can be seen that the effect of the number of iterations has a large impact on this example. The iterations at 10000 is not sufficient to fit the boundary well.

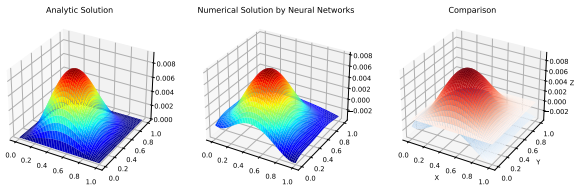
2. Compare PDE loss: it is obvious in figure 14 that the PDE loss is decreasing from less than $1e-3$ to less than $1e-5$ as the iteration increases from 10000 to 50000.

3. Compare L_2 error: shown in figure 15, the L_2 error decreases as the number of iterations increases, however, the magnitude of the L_2 error oscillation increases at the same time and it is difficult to reach the $1e-3$ until the iteration = 50000. This shows that the capability of the model is not very stable

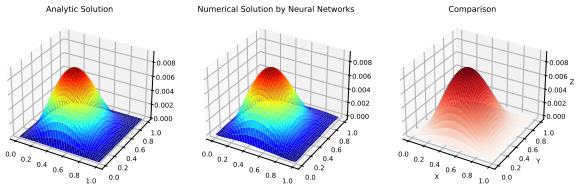
The Table 4 records the results at different iterations.

5.3.2 Training data sets

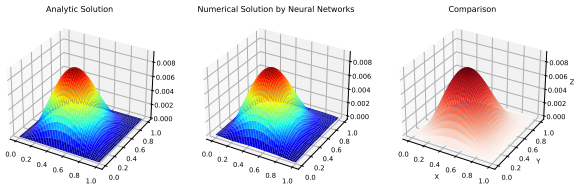
We then explore whether the size of the training set will have an impact on the results, and record the losses in figure 16 for different combinations of boundary and interior points.



(a) iteration = 10, 000

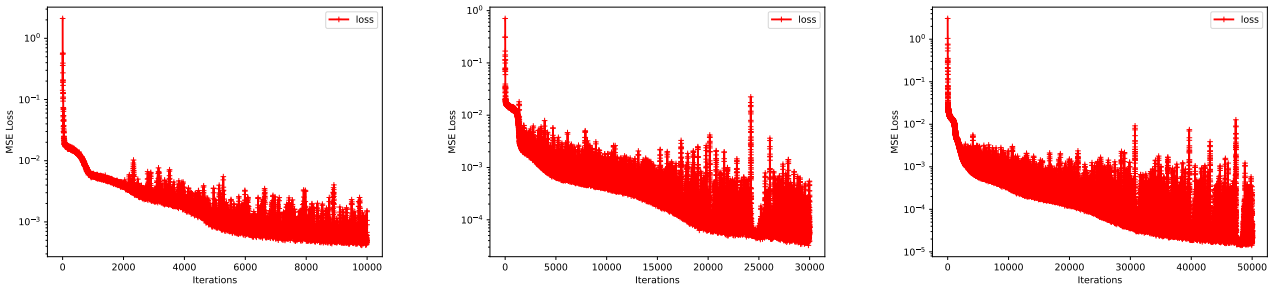


(b) iteration = 30, 000



(c) iteration = 50, 000

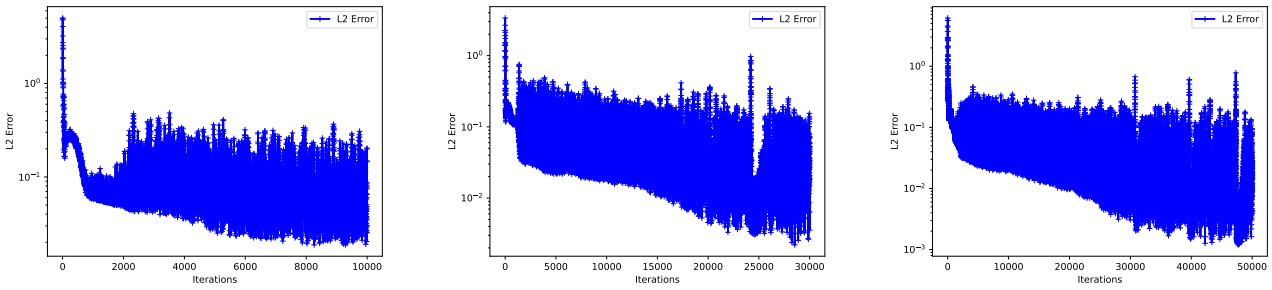
Figure 13: Comparison of Numerical and Analytical Solution



(a) iteration = 10, 000 (b) iteration = 30, 000 (c) iteration = 50, 000

Figure 14: Comparison of PDE Loss

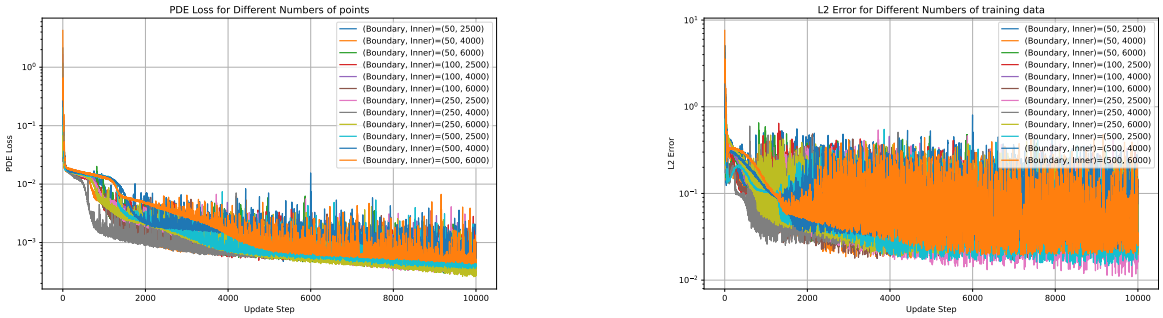
1. PDE loss 16a: the combination $(N_b = 250, N_f = 6000)$ demonstrates the best outcome to minimize the PDE loss. Meanwhile, comparing the PDE loss of $(N_b = 500, N_f = 6000)$, we can obtain the conclusion that expanding the size of the training set will not necessarily lead to the greatest decrease in loss, which is also the same as Neumann example shown in figure 10a.



(a) iteration = 10, 000 (b) iteration = 30, 000 (c) iteration = 50, 000

Figure 15: Comparison of L_2 Error

2. L_2 error 16b: the oscillation of the losses for different sizes of training data-set exhibits a sight variations, $(N_b = 250, N_f = 2500)$ seems to fit the analytical solution the best. Nevertheless, the oscillatory trends are more similar as the number of iterations increases, suggesting that the size does not have an obvious impact on the ability of the model.



(a) PDE loss of different numbers of points (b) L_2 error of different numbers of points

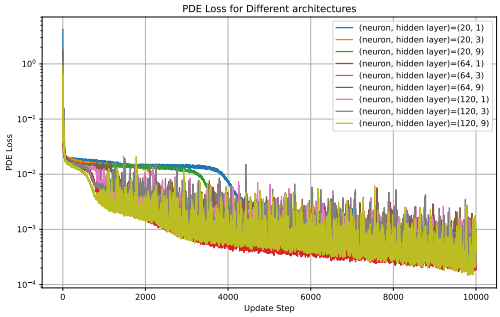
Figure 16: Compare Losses of Different Sizes of Data-Set

5.3.3 Architecture

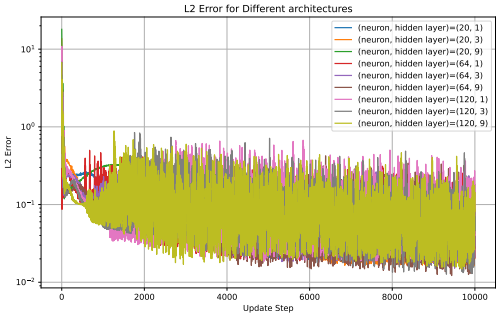
Finally, we vary the number of neurons as well as the number of hidden layers to compare their results.

1. The PDE losses in figure 17a: when the number of neurons is smaller at 20, the loss is harder to decrease.

2. The L_2 error in figure 17b: the oscillatory trends of the L_2 errors tend to be similar, indicating that the architecture does not affect the performance of the model.



(a) PDE loss of different architectures



(b) L_2 error of different architectures

Figure 17: Compare Losses of Different Architectures

5.3.4 Weights

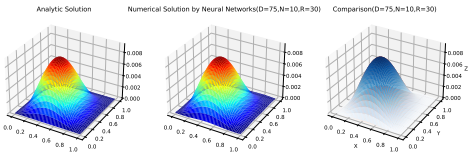
Different combinations of weights are tested to evaluate the performance at the iteration = 40000, and Table 5 records the results, which are ranked in order of error from smallest to largest. It is more intuitive to compare the numerical and analytical solution of ($D_{\text{weight}} = 75, N_{\text{weight}} = 10, R_{\text{weight}} = 30$) with the least run time **295** seconds and ($D_{\text{weight}} = 50, N_{\text{weight}} = 20, R_{\text{weight}} = 30$). The former figure 18a almost fits all boundaries at 40000 iterations but the latter figure 18b fails to fit the boundary.

Weights(Dirichlet, Neumann, Robin)	Error	Runtime
(75, 10, 30)	1.82E-03	295.229
(75, 20, 20)	4.92E-03	347.029
(100, 20, 30)	6.97E-03	362.708
(100, 20, 20)	1.07E-02	348.271
(75, 30, 10)	1.21E-02	336.896
(100, 30, 30)	1.22E-02	309.025
(75, 30, 30)	1.23E-02	327.057
(75, 10, 20)	1.24E-02	309.694
(50, 10, 10)	1.28E-02	351.954
(50, 20, 30)	7.89E-02	327.607

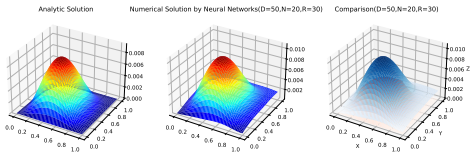
Table 5: Comparison of Weights of Boundaries

6 Confusions of Details

Over the past two months, we have encountered a variety of issues. Fortunately, we had the opportunity to communicate with outstanding seniors, which allowed us to resolve many detailed problems effectively. This is indeed a precious experience.



(a) $(D_{\text{weight}} = 75, N_{\text{weight}} = 10, R_{\text{weight}} = 30)$



(b) $(D_{\text{weight}} = 50, N_{\text{weight}} = 20, R_{\text{weight}} = 30)$

Figure 18: Comparison of Numerical and Analytical Solution

1. Why PINNs often fail to train: at first, we doubted whether it was reasonable to adjust parameters so frequently. However, the thesis[3] provided us with the theoretical support, and the experimental results in both the Neumann example in section 5.2.4 and the Robin example in section 5.3.4 have shown that the most significant factor affecting the performance of a model is the weights of the boundary loss.

2. The location of generating training data set: Initially, we were uncertain whether it was more effective to generate points for the training set in advance and then pass them into the training as the full batch, or to randomly generate new training points at each iteration during the training process. Eventually, we chose the latter approach, as the paper[1] mentioned that they selected to re-sampled the collocation points in each epoch to get better coverage of the sampling domain.

3. Learning rate: initially, our error fluctuations were quite large. Subsequently, we learned that one of the reasons for this was the high learning rate of our neural network. By reducing the learning rate, we could achieve greater precision in our learning, although this often required more iterations.

7 Limitations

1. The results clearly show that the error still exhibits significant fluctuations, indicating that the model is relatively unstable.

2. We have only assessed the impact of a limited number of factors on the model. Other factors, such as the option of the loss function, activation function, optimizer, etc., could also potentially affect the outcomes.

References

[1] Tamara G Grossmann, Urszula Julia Komorowska, Jonas Latz, and Carola-Bibiane Schönlieb. Can physics-informed neural networks beat the finite element method? *arXiv preprint arXiv:2302.04107*, 2023.

[2] M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[3] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why pinns fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022.

[4] Wikipedia contributors. Physics-informed neural networks — Wikipedia, the free encyclopedia. `https://en.wikipedia.org/w/index.php?title=Physics-informed_neural_networks&oldid=1187284467`, 2023. [Online; accessed 10-December-2023].